

Trabajo Práctico N° 2 — Expo Router: rutas, navegación, pilas y colas

Alumno/a:		Fecha de entrega:	
-----------	--	-------------------	--

Consignas generales

Objetivo: aplicar la teoría de Expo Router (SDK 57) para crear una app con varias pantallas, pasar datos entre ellas y proteger rutas, entendiendo cómo la **pila** y la **cola** explican lo que pasa al navegar.

Modalidad: las Partes A a F son de preguntas y ejercicios cortos; respondelas en un archivo `RESPUESTAS.md` dentro del repositorio del proyecto. La Parte G es el desarrollo del sistema. Todo se entrega en un único repositorio.

Material de consulta: la presentación de clase “Expo Router” y la documentación oficial (docs.expo.dev/routing/introduction).

Parte A · Estructuras de datos: la pila y la cola

A1. Conceptos

Respondé con tus palabras:

- ¿Qué significan **LIFO** y **FIFO**? ¿Cuál corresponde a la pila y cuál a la cola?
- ¿Por qué extremo entra y por qué extremo sale un elemento en cada estructura?
- Dá un ejemplo de la vida real y otro de una aplicación móvil para cada una.

A2. Seguimiento de una pila

Usando la clase `Pila` vista en clase (`push`, `pop`, `tope`, `vacía`), indicá qué imprime cada `console.log` y cómo queda la pila al final (de base a tope).

```
seguimiento-pila.js
const p = new Pila();
p.push('Inicio');
p.push('Productos');
p.push('Detalle 3');
p.pop();
p.push('Perfil');
console.log(p.tope()); // (1)
console.log(p.pop()); // (2)
console.log(p.tope()); // (3)
console.log(p.vacía); // (4)
```

A3. Seguimiento de una cola

Ahora con la clase `Cola` (`encolar`, `desencolar`, `frente`, `vacía`). Indicá las salidas y el estado final (de frente a final).

```
seguimiento-cola.js
const c = new Cola();
c.encolar('Ana');
c.encolar('Beto');
c.desencolar();
c.encolar('Caro');
c.encolar('Dani');
console.log(c.frente()); // (1)
console.log(c.desencolar()); // (2)
console.log(c.vacia); // (3)
```

A4. Análisis de la implementación

- En las clases de clase, el array se declara como `#items`. ¿Qué significa el `#` y qué problema evita?
- La cola usa `array.shift()` para desencolar. ¿Qué problema de rendimiento tiene con colas muy grandes? ¿Cómo lo resuelven las colas “serias”?
- ¿Qué método de array usa la pila para sacar y cuál usa la cola? ¿Por qué no pueden usar el mismo?

A5. Programación: una cola eficiente

Implementá en JavaScript la clase `ColaEficiente` con los métodos `encolar(x)`, `desencolar()`, `frente()` y los getters `vacía` y `tamaño`, **sin usar** `shift()`. Pista: guardá en un campo privado el índice del frente.

A6. Pila y cola dentro de Expo Router

- ¿Qué estructura describe el **historial de pantallas** de un Stack? ¿Qué pantalla es la visible y qué operación hace “atrás”?
- ¿Qué estructura usa Expo Router para las **acciones de navegación**? ¿Qué pasa si el usuario toca dos links muy rápido?

Parte B · Rutas basadas en archivos

B1. Del archivo a la URL

Completá la tabla. Si el archivo no genera una pantalla, explicá qué hace. Si genera un problema, indicalo.

Archivo	URL que genera / función
<code>src/app/(tabs)/index.tsx</code>	
<code>src/app/acerca.tsx</code>	
<code>src/app/(tabs)/perfil.tsx</code>	
<code>src/app/(tabs)/productos/index.tsx</code>	
<code>src/app/(tabs)/productos/[id].tsx</code>	
<code>src/app/docs/[...slug].tsx</code>	
<code>src/app/_layout.tsx</code>	
<code>src/app/+not-found.tsx</code>	
<code>src/app/Boton.tsx</code>	

B2. De la URL al archivo

Indicá qué archivo (ruta completa dentro de `src/app`) tenés que crear para que existan estas URLs:

URL	Archivo
<code>/categorias/bebidas</code> (y cualquier otra categoría)	
<code>/buscar?q=mate&categoria=kiosco</code>	
<code>/ayuda/pagos/tarjeta</code> y <code>/ayuda/horarios</code>	
<code>/ayuda</code> (con una pantalla propia)	

B3. Verdadero o falso

Indicá V o F y **justificá** las falsas.

- Con Expo Router, cada pantalla nueva se debe registrar en una tabla de configuración.
- Los archivos `_layout.tsx` son pantallas que el usuario puede visitar.
- Una carpeta entre paréntesis, como `(tabs)`, no aparece en la URL.
- Para agregar una librería conviene usar `npm install`, porque siempre trae la última versión.
- En `package.json`, `"main": "expo-router/entry"` reemplaza al viejo `App.tsx`.
- La ruta `/_sitemap` lista todas las rutas de la app y sirve para depurar.
- Si existen `docs/index.tsx` y `docs/[...slug].tsx`, la URL `/docs` muestra `docs/index.tsx`.
- En SDK 57, `expo-router` usa el mismo número de versión mayor que el SDK (57).

Parte C · Navegar: <Link>, router y la pila

C1. Métodos de router

Completá qué le hace cada método a la pila del Stack.

Método	Qué le hace a la pila
<code>router.push(href)</code>	
<code>router.navigate(href)</code>	
<code>router.replace(href)</code>	
<code>router.back()</code>	
<code>router.dismissTo(href)</code>	
<code>router.dismissAll()</code>	
<code>router.canGoBack()</code>	
<code>router.setParams({...})</code>	

C2. Simulación de la pila

Suponé un **único Stack** que contiene todas estas pantallas. La pila empieza en [`/productos`].
Escribí cómo queda la pila (de base a tope) después de cada instrucción.

#	Instrucción	Pila resultante
1	<code>router.push("/productos/1")</code>	
2	<code>router.push("/productos/2")</code>	
3	<code>router.navigate("/productos/5")</code>	
4	<code>router.push("/perfil")</code>	
5	<code>router.replace("/buscar")</code>	
6	<code>router.back()</code>	
7	<code>router.dismissTo("/productos")</code>	
8	<code>router.canGoBack()</code> → ¿qué devuelve?	

C3. ¿Link o router?

Para cada situación, elegí `<Link>` o `router` e indicá el método o prop que usarías. Justificá.

- El usuario toca la tarjeta de un producto en una lista.
- Se guarda un formulario, la API responde OK y hay que mostrar la pantalla de éxito.
- Botón "Cancelar" dentro de un modal.
- Después de un login exitoso hay que ir a la pantalla principal.
- Volver desde el detalle de un pedido directamente a la lista de pedidos, que quedó tres pantallas más abajo.

C4. Escribí el código

- Un `<Link>` que abra el producto con id 8 usando **href como objeto**.
- Un `<Link>` a `/perfil` que **siempre** apile, aunque la pantalla ya exista.
- Un botón (`Pressable`) propio que funcione como link a `/carrito` usando `asChild`.

C5. Pensar

En una web, cada `<Link>` se convierte en un `<a href>` real. ¿Qué ventaja concreta tiene eso para el usuario? ¿Qué pasa en el celular, donde no hay barra de direcciones?

Parte D · Navegadores: Stack, Tabs y Drawer

D1. Comparación

Completá la tabla.

	Stack	Tabs	Drawer
¿Apila pantallas?			
¿Cómo cambia de pantalla el usuario?			
¿Desde dónde se importa en SDK 57?			
Un caso de uso típico			

D2. Cada tab tiene su pila

En una app con pestañas Inicio y Productos (Productos tiene su propio Stack), el usuario está en Productos, abre el detalle del producto 4, cambia a Inicio y vuelve a Productos. ¿Qué pantalla ve? ¿Por qué? ¿Qué app que uses todos los días se comporta así?

D3. ¿Dónde va cada pantalla?

Aplicando la regla práctica de navegadores anidados, indicá si cada pantalla va en el **Stack raíz** o **dentro de una tab**:

- a) El detalle de un producto, que debe mantener visible la barra de pestañas.
- b) Un modal para confirmar una compra, que debe tapar la barra de pestañas.
- c) La pantalla de login que se abre como modal.
- d) La pantalla “Mis pedidos anteriores” dentro de la sección Perfil.

D4. Configurar el Stack

Observá el layout y respondé:

```
src/app/_layout.tsx
import { Stack } from 'expo-router';
export default function LayoutRaiz() {
  return (
    <Stack screenOptions={{ headerBackButtonDisplayMode: 'minimal' }}>
      <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
      <Stack.Screen name="modal" options={{ presentation: 'modal' }} />
      <Stack.Screen
        name="hoja"
        options={{ presentation: 'formSheet', sheetAllowedDetents: [0.5, 0.9] }}
      />
    </Stack>
  );
}
```

- a) ¿Qué diferencia hay entre `screenOptions` y las `options` de un `Stack.Screen`?
- b) ¿Por qué `(tabs)` tiene `headerShown: false`?
- c) Si existe `src/app/perfil-publico.tsx` pero no está declarada en el Stack, ¿existe la pantalla? ¿Para qué sirve declararla?
- d) Nombrá cuatro valores posibles de `presentation`. ¿Cuál usarías para una hoja inferior que se abre al 50%?
- e) ¿Cómo cambiarías el título del header desde la propia pantalla de detalle para que diga “Producto 7”?

D5. Tabs y Drawer en SDK 57

- a) ¿Qué cambió en SDK 57 al importar `Tabs`? ¿Qué alternativa experimental existe?
- b) ¿Qué dos paquetes necesita el Drawer y qué componente conviene poner en el layout raíz para los gestos?
- c) ¿Hace falta instalar `@react-navigation/drawer` en SDK 57? ¿Por qué?
- d) Si hay navegadores anidados, ¿en qué navegador actúa `router.back()`?

Parte E · Rutas dinámicas, parámetros y hooks

E1. Encontrá el error

Los productos tienen `id` numérico (`{ id: 3, nombre: "Chipá" }`). La pantalla nunca encuentra el producto. Explicá por qué y corregilo.

```
src/app/(tabs)/productos/[id].tsx
export default function DetalleProducto() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const producto = productos.find((p) => p.id === id);
  if (id === 3) console.log('Es el chipá');
  if (!producto) return <Text>No existe el producto {id}</Text>;
  return <Text>{producto.nombre}</Text>;
}
```

E2. Catch-all

Para `src/app/docs/[...slug].tsx`, indicá el valor de `slug` en cada caso:

URL	slug
<code>/docs/react</code>	
<code>/docs/react/hooks/useState</code>	
<code>/docs</code>	

E3. Anatomía de una URL

Dada la URL `rutasipf://buscar?q=mate&categoria=bebidas`:

- Identificá el scheme, la ruta y los parámetros de búsqueda.
- ¿Qué devuelve `useLocalSearchParams()` en `buscar.tsx`?
- ¿Hacen falta corchetes en el nombre del archivo para recibir `q`? ¿Por qué?
- En el buscador, cada vez que el usuario escribe se llama a `router.setParams({ q: texto })` en lugar de `router.push`. Dá dos razones.

E4. ¿Dónde estoy?

Completá los valores de cada hook en las dos URLs de la app de ejemplo (`buscar.tsx` está en el Stack raíz; el detalle está en `(tabs)/productos/[id].tsx`).

Hook	En <code>/productos/3</code>	En <code>/buscar?q=chipa</code>
<code>usePathname()</code>		
<code>useSegments()</code>		
<code>useLocalSearchParams()</code>		

E5. Local vs global

- ¿Cuál es la diferencia entre `useLocalSearchParams` y `useGlobalSearchParams`? ¿Cuál es la opción por defecto y por qué?
- ¿Para qué sirve `useFocusEffect`? Dá un ejemplo de uso.
- La URL `/productos/mate` abre la pantalla de detalle aunque no exista ese producto. ¿Es un error de Expo Router? ¿De quién es la responsabilidad?

Parte F · Redirecciones, rutas protegidas y deep links

F1. Redirect

- a) ¿Qué hace `<Redirect href="/productos" />` y a qué método de `router` equivale?
- b) ¿Por qué una redirección debe **reemplazar** y no **apilar**? Describí el problema que aparecería.

F2. Stack.Protected

Completá los `guard` para que `privado` solo exista con sesión y `login` solo sin sesión. Luego respondé.

```
src/app/_layout.tsx
function NavegacionRaiz() {
  const { usuario } = useAuth();
  const conSesion = usuario !== null;
  return (
    <Stack>
      <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
      <Stack.Protected guard={ _____ }>
        <Stack.Screen name="privado" />
      </Stack.Protected>
      <Stack.Protected guard={ _____ }>
        <Stack.Screen name="login" options={{ presentation: 'modal' }} />
      </Stack.Protected>
    </Stack>
  );
}
```

- a) ¿Qué le pasa a una pantalla cuando su `guard` es `false`?
- b) Al iniciar sesión, el modal de login se cierra solo, sin llamar a `router.back()`. ¿Por qué?
- c) Aparece el aviso "The action 'NAVIGATE' ... was not handled by any navigator". ¿Qué lo causa y cómo se evita?
- d) ¿Qué ventaja tiene `Stack.Protected` frente a poner un `<Redirect>` condicional en cada pantalla?

F3. 404, anchor y rutas tipadas

Explicá brevemente para qué sirve cada uno y en qué archivo se define:

- a) `+not-found.tsx`
- b) `export const unstable_settings = { anchor: "(tabs)" }`
- c) `typedRoutes`: ¿qué pasa si escribís `<Link href="/prodcutos" />`? ¿Dónde se generan los tipos?

F4. Deep links

La app tiene `"scheme": "comedoripf"` en `app.json` y la compu de desarrollo tiene la IP `192.168.1.20`. Escribí la URL que abre el plato 7 (`/menu/7`) en cada caso:

Dónde	URL
App instalada (build propia)	
Expo Go en desarrollo	
Web (<code>npx expo start --web</code>)	

¿Qué significa la parte `/--/` en la URL de Expo Go? ¿Por qué el scheme propio no funciona dentro de Expo Go?

F5. Errores comunes

Para cada situación explicá la causa y la solución:

- Al usar `<Link href="/perfil" asChild>` con un `<Pressable style={[estilos.boton, activo && estilos.activo]}>` aparece: "You are passing an array of styles to a child of `<Slot>`".
- Un compañero creó `src/app/TarjetaProducto.tsx` para reutilizar un componente y ahora la app tiene una ruta nueva.
- Después de iniciar sesión se usa `router.push("/")` y, al tocar atrás, el usuario vuelve al login.
- Expo Go dice que el proyecto es incompatible después de instalar un paquete con `npm install`.

Parte G · Desarrollo del sistema: "Comedor IPF"

El comedor del Instituto quiere una app para que los alumnos pidan comida desde el celular y la cocina atienda los pedidos **en orden de llegada**. Vas a desarrollarla con **Expo Router (SDK 57)** aplicando todo lo visto. El sistema combina las dos estructuras de la clase: una **Cola** para los pedidos y una **Pila** para deshacer acciones en el carrito.

G1. Rutas requeridas

La app debe responder exactamente a estas URLs. **Parte del trabajo es diseñar el árbol de carpetas** que las genera (incluílo en el README).

URL	Pantalla	Navegador / detalle
/	Inicio: saludo y tarjetas de acceso rápido (Menú, Buscar, Ayuda, Cocina).	Tab "Inicio".
/menu	Lista de platos agrupados por categoría.	Tab "Menú", con Stack propio.
/menu/[id]	Detalle del plato: nombre, precio, descripción y botón "Agregar al carrito". Título del header = nombre del plato.	Dentro de la tab Menú (la barra de pestañas sigue visible).
/categorias/[categoria]	Platos de una categoría: desayuno, almuerzo, bebidas o kiosco.	Stack raíz.
/buscar?q=&categoria=	Buscador por texto y filtro por categoría. La búsqueda vive en la URL.	Stack raíz.
/carrito	Ítems agregados, total, botón "Deshacer último" y botón "Confirmar pedido". Badge con la cantidad de ítems.	Tab "Carrito", con Stack propio.
/carrito/nota	Aclaración para la cocina (ej.: "sin sal").	Dentro de la tab Carrito.
/confirmar	Resumen del pedido y botón "Confirmar".	Stack raíz, <code>presentation: "modal"</code> .
/turno/[numero]	Número de turno y cuántos pedidos hay adelante en la cola.	Stack raíz.
/login	Ingreso del personal de cocina (usuario y clave fijos en el código).	Modal, solo existe sin sesión.
/cocina	Pedido del frente de la cola, cantidad en espera y botón "Atender siguiente".	Drawer, solo existe con sesión.
/cocina/atendidos	Pedidos ya atendidos, del más reciente al más antiguo.	Misma sección del Drawer.
/ayuda y /ayuda/...	Índice de ayuda y artículos de profundidad variable (ej.: <code>/ayuda/pagos/efectivo</code>).	Pantalla índice + catch-all.
/pedido	URL vieja de la versión anterior: debe llevar a <code>/carrito</code> .	<code><Redirect></code> .

URL	Pantalla	Navegador / detalle
cualquier otra	Pantalla 404 que muestre la URL inexistente.	+not-found.tsx.

G2. Requisitos funcionales

1. **Estructuras de datos propias.** En `src/estructuras/` implementá las clases `Pila` y `Cola` con campos privados (`#items`). La cola no puede usar `shift()` (reutilizá lo resuelto en A5). Agregá a ambas un getter `tamano` y un método `aArray()` que devuelva una copia.
2. **Cola de pedidos.** Al confirmar un pedido se le asigna un número correlativo y se **encola**. La pantalla de cocina muestra el `frente()` y el botón “Atender siguiente” **desencola**. Nadie puede “colarse”.
3. **Pila de deshacer.** Cada vez que se agrega un plato al carrito se hace `push` en una pila de acciones. “Deshacer último” hace `pop` y quita ese plato. El botón se deshabilita cuando la pila está vacía.
4. **Historial de atendidos.** Cada pedido atendido se apila en otra `Pila`; `/cocina/atendidos` la muestra del tope a la base (el último atendido primero).
5. **Navegación correcta.** Usá `<Link>` cuando el usuario toca algo y `router` cuando navegás después de una lógica. Al confirmar, pasá de `/confirmar` a `/turno/[numero]` de forma que “atrás” **no** vuelva a la confirmación (justificá el método elegido en el README).
6. **Rutas dinámicas y validación.** `/menu/[id]` y `/categorias/[categoria]` deben validar el parámetro (recordá que llega como texto) y mostrar un mensaje si no existe.
7. **Buscador con parámetros.** `/buscar` lee `q` y `categoria` con `useLocalSearchParams` y los actualiza con `router.setParams`, sin apilar pantallas. Una búsqueda debe poder compartirse con un link.
8. **Rutas protegidas.** `cocina` y `login` se controlan con `Stack.Protected` en el layout raíz. Al iniciar sesión, el login se cierra solo; al cerrar sesión desde la cocina, la sección desaparece del historial.
9. **Estado global.** La sesión, el carrito, la cola y las pilas viven en un Context cuyo provider se coloca en el layout raíz.
10. **Componente “¿Dónde estoy?”.** Creá `src/components/DondeEstoy.tsx` que muestre `usePathname()`, `useSegments()` y `useLocalSearchParams()`, y agregalo al final de cada pantalla (se puede ocultar con una constante `DEBUG`).

G3. Requisitos técnicos

- Proyecto creado con `npx create-expo-app@latest`, Expo SDK 57, TypeScript. Paquetes instalados **solo** con `npx expo install`.
- Tabs importadas desde `expo-router/js-tabs`, con íconos de `@expo/vector-icons`. Drawer desde `expo-router/drawer`, con `GestureHandlerRootView` en el layout raíz.
- En `src/app` **solo rutas**: componentes en `src/components`, datos en `src/data`, estructuras en `src/estructuras`, contexto en `src/context`.
- `"scheme": "comedoripf"` configurado; `anchor` en (`tabs`) para que un deep link a `/categorias/bebidas` deje las pestañas debajo en la pila.
- Rutas tipadas activas: ningún `href` escrito a mano puede tener errores de TypeScript.
- Al menos 12 platos de ejemplo en `src/data/platos.ts`, repartidos en las cuatro categorías.

G4. Desafíos opcionales (suman puntos extra)

- **Contador de pila:** mostrar en el título del header cuántas pantallas hay en la pila, usando `useNavigation().getState()`.
- **Tab protegida:** con `Tabs.Protected`, que aparezca una pestaña “Cocina” solo con sesión iniciada.

- **Hoja inferior:** mostrar la nota del carrito como `presentation: "formSheet"` con `sheetAllowedDetents`.
- **Tiempo estimado:** en `/turno/[numero]`, calcular la espera multiplicando la posición en la cola por 3 minutos.

G5. Entregables

1. Repositorio con el proyecto funcionando (sin `node_modules`).
2. `RESPUESTAS.md` con las respuestas de las Partes A a F.
3. `README.md` con: árbol de carpetas de `src/app` y el navegador de cada `_layout`, justificación de `replace` vs `push` en el flujo de confirmación y capturas (o un video corto) de: carrito con deshacer, turno, cocina atendiendo pedidos, login/logout y la pantalla 404.
4. Un deep link de prueba que abra un plato directamente en Expo Go (anotalo en el README).

G6. Preguntas para la defensa oral

En la defensa del trabajo se te pedirá explicar, sobre tu propio código:

1. ¿Qué método usaste para pasar de `/confirmar` a `/turno/[numero]` y qué pasaría si usaras `push`?
2. Si la cocina cierra sesión estando en `/cocina/atendidos`, ¿qué ocurre con esa pantalla? ¿Por qué no hace falta llamar a `router.back()`?
3. ¿Por qué "Deshacer" usa una pila y no una cola? ¿Y por qué los pedidos usan una cola y no una pila?
4. ¿Qué pasa si alguien abre `comedoripf://menu/999`? ¿Y `comedoripf://no-existe`?
5. Si tocás "Agregar al carrito" y enseguida el link al carrito, ¿en qué orden se procesan esas acciones de navegación y por qué?
6. ¿Qué pantalla queda debajo cuando se abre `/categorias/bebidas` desde un deep link? ¿Qué configuración lo decide?

G7. Criterios de evaluación

Criterio	Puntos
Partes A a F (preguntas y ejercicios)	40
Árbol de rutas y layouts correctos (grupos, Stack dentro de tabs, modal, Drawer)	15
Navegación: uso adecuado de Link / router, replace justificado, dismissTo / back	10
Rutas dinámicas, catch-all y parámetros de búsqueda con validación	10
Pila y Cola propias, correctas y usadas en el sistema	10
Rutas protegidas, Redirect y 404	5
Organización del código, README y defensa oral	10
Total	100
Desafíos opcionales	hasta +10